

ARTS Deployment Procedure

Ubuntu server, Hostinger domain, Cloudflare, Nginx reverse proxy, Flask/Gunicorn, PostgreSQL, and Synology backup/storage

1. Target Architecture

- Users access <https://yourdomain.com> through Cloudflare.
- Cloudflare proxies traffic to your public static IP and provides DNS, SSL edge protection, DDoS protection, and security rules.
- Your router forwards only TCP 80 and 443 to the Ubuntu server.
- Nginx receives public traffic and reverse-proxies requests to Gunicorn on 127.0.0.1:5000.
- Gunicorn runs the Flask ARTS application from app:app.
- PostgreSQL stores application data. The current system stores applicant PDFs and profile images inside PostgreSQL binary columns.
- Synology receives scheduled database backups. Optional future app changes can store uploaded PDFs directly on a Synology-mounted share.

2. Things To Buy Or Prepare

Item	Purpose
Domain from Hostinger	Public name for the ARTS system.
Static public IPv4 from ISP	Required for reliable self-hosting from your custom Ubuntu PC.
Cloudflare Free or Pro	DNS, CDN, DDoS protection, SSL, WAF/rate controls. Start Free; consider Pro for heavier traffic.
UPS battery backup	Keeps Ubuntu server, router, modem, and Synology online during short power loss.
Synology NAS storage	Local backup target and optional uploaded-file storage.
Offsite backup storage	Recommended second backup copy: Backblaze B2, AWS S3, Wasabi, or Synology C2.
Optional firewall/router	pfSense, OPNsense, MikroTik, or UniFi for stronger network control.

3. Ubuntu Server Setup

- 1 Log in to the Ubuntu server with SSH.
- 2 Update the server and install Python, PostgreSQL, Nginx, UFW, Fail2ban, Git, and backup tools.
- 3 Create a dedicated Linux user named ats.
- 4 Place the project in /home/ats/app.
- 5 Create a production .env file with SECRET_KEY and DATABASE_URL.
- 6 Create a Python virtual environment and install requirements plus gunicorn.
- 7 Test Gunicorn locally before exposing the site.

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3 python3-venv python3-pip \
    postgresql postgresql-contrib nginx ufw fail2ban \
    git curl unzip rsync postgresql-client
sudo adduser ats
sudo usermod -aG sudo ats
su - ats
mkdir -p /home/ats/app
cd /home/ats/app
python3 -m venv .env
source .env/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

```
pip install gunicorn
```

4. PostgreSQL Setup

- Create the ats database and a non-admin user named ats_user.
- Use a long random password.
- Do not expose PostgreSQL port 5432 to the internet.

```
sudo -u postgres psql
CREATE DATABASE ats;
CREATE USER ats_user WITH ENCRYPTED PASSWORD 'REPLACE_WITH_LONG_RANDOM_DB_PASSWORD';
GRANT ALL PRIVILEGES ON DATABASE ats TO ats_user;
\q
sudo -u postgres psql -d ats
GRANT ALL ON SCHEMA public TO ats_user;
ALTER SCHEMA public OWNER TO ats_user;
\q
```

5. Production Environment File

```
cd /home/ats/app
nano .env

SECRET_KEY=REPLACE_WITH_LONG_RANDOM_SECRET_KEY
DATABASE_URL=postgresql://ats_user:REPLACE_WITH_LONG_RANDOM_DB_PASSWORD@localhost:5432/ats
FLASK_ENV=production
```

6. Gunicorn Systemd Service

```
sudo nano /etc/systemd/system/ats.service

[Unit]
Description=Applicant Tracking System Flask App
After=network.target postgresql.service

[Service]
User=ats
Group=www-data
WorkingDirectory=/home/ats/app
EnvironmentFile=/home/ats/app/.env
ExecStart=/home/ats/app/.venv/bin/gunicorn -w 4 -b 127.0.0.1:5000 app:app
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable ats
sudo systemctl start ats
sudo systemctl status ats
```

7. Nginx Reverse Proxy

Reverse proxy meaning: Nginx is the public front door on ports 80/443. It forwards requests internally to Flask/Gunicorn on 127.0.0.1:5000. This keeps Flask off the public internet and lets Nginx handle HTTPS, static files, rate limiting, and buffering.

```
sudo nano /etc/nginx/sites-available/ats

server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;
    client_max_body_size 5M;

    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    location /static/ {
        alias /home/ats/app/static/;
        expires 7d;
        add_header Cache-Control "public";
    }

    location / {
```

```

        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

sudo ln -s /etc/nginx/sites-available/ats /etc/nginx/sites-enabled/ats
sudo nginx -t
sudo systemctl reload nginx

```

8. Firewall And Router

- Ubuntu UFW: allow only OpenSSH, 80/tcp, and 443/tcp.
- Router port forwarding: forward public TCP 80 and 443 only to the Ubuntu server LAN IP.
- Do not forward Flask 5000, PostgreSQL 5432, Synology DSM 5000/5001, SMB 445, rsync 873, or NFS 2049.

```

sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose

```

9. Hostinger Domain And Cloudflare DNS

- 1 In Hostinger hPanel, go to Domains > Get a new domain, buy the domain, enable privacy if available, and enable auto-renewal.
- 2 In Cloudflare, click Add a site, enter the domain, and choose Free or Pro.
- 3 In Cloudflare DNS, add an A record for @ pointing to your static public IPv4. Set proxy status to Proxied.
- 4 Add a CNAME record for www pointing to your root domain. Set proxy status to Proxied.
- 5 Cloudflare will show two nameservers.
- 6 In Hostinger hPanel, go to Domains > Domain portfolio > Manage > DNS / Nameservers > Change Nameservers.
- 7 Enter the two Cloudflare nameservers and save. Wait for propagation, usually minutes to 24 hours.

Type	Name	Target	Proxy
A	@	YOUR_PUBLIC_STATIC_IP	Proxied
CNAME	www	yourdomain.com	Proxied

10. HTTPS Certificate

```

sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
sudo certbot renew --dry-run

```

- In Cloudflare SSL/TLS, set mode to Full (strict).
- Enable Always Use HTTPS.
- Confirm https://yourdomain.com loads before announcing the system.

11. Synology Backup Configuration

- 1 In Synology DSM, open Control Panel > Shared Folder > Create > Create Shared Folder.
- 2 Create a shared folder named ats_backups.
- 3 Enable Recycle Bin. Enable encryption if your Synology supports it.
- 4 Open Control Panel > User & Group > Create.

- 5 Create a non-admin user named ats_backup.
- 6 Give ats_backup Read/Write access only to ats_backups and No Access to other folders.
- 7 Open Control Panel > File Services > rsync and enable rsync service.
- 8 Open Control Panel > Application Privileges > rsync > Edit and allow ats_backup. Restrict to the Ubuntu server LAN IP if available.

```
nano /home/ats/backup_to_synology.sh

#!/bin/bash
set -e
DATE=$(date +%F-%H%M)
LOCAL_BACKUP_DIR="/var/backups/ats"
DB_URL="postgres://ats_user:YOUR_DB_PASSWORD@localhost:5432/ats"
SYNOLOGY_IP="192.168.1.20"
SYNOLOGY_USER="ats_backup"
SYNOLOGY_TARGET="ats_backups"
mkdir -p "$LOCAL_BACKUP_DIR"
pg_dump "$DB_URL" | gzip > "$LOCAL_BACKUP_DIR/ats-db-$DATE.sql.gz"
rsync -avz "$LOCAL_BACKUP_DIR/" "$SYNOLOGY_USER@$SYNOLOGY_IP:/$SYNOLOGY_TARGET/"
find "$LOCAL_BACKUP_DIR" -type f -name "*.gz" -mtime +7 -delete

chmod +x /home/ats/backup_to_synology.sh
/home/ats/backup_to_synology.sh
crontab -e
0 2 * * * /home/ats/backup_to_synology.sh >> /home/ats/backup.log 2>&1
```

12. Uploaded PDFs And Excel Files

- Current behavior: applicant CV PDFs are stored in PostgreSQL as applicant.cv_data.
- Current behavior: employee profile images are stored in PostgreSQL as employee.profile_data.
- Current behavior: Excel exports are generated in memory and downloaded; they are not permanently stored by the app.
- Because PDFs are currently inside PostgreSQL, the Synology database backup includes the PDFs.
- Optional future improvement: change the app to store uploads on /mnt/ats_uploads and keep only file path, filename, content type, size, and checksum in PostgreSQL.

13. Optional Direct Synology Upload Storage

- 1 In DSM, create another shared folder named ats_uploads.
- 2 Create a non-admin user named ats_storage.
- 3 Give ats_storage Read/Write access only to ats_uploads.
- 4 On Ubuntu, mount the Synology share at /mnt/ats_uploads using CIFS.
- 5 Only after the mount is reliable, modify the Flask app to write PDFs/profile images to that mount instead of database binary columns.

```
sudo apt install -y cifs-utils
sudo mkdir -p /mnt/ats_uploads
sudo nano /etc/ats-synology-credentials

username=ats_storage
password=YOUR_STRONG_SYNOLOGY_PASSWORD

sudo chmod 600 /etc/ats-synology-credentials
sudo nano /etc/fstab

//192.168.1.20/ats_uploads /mnt/ats_uploads cifs \
credentials=/etc/ats-synology-credentials,uid=ats,gid=www-data,\
file_mode=0640,dir_mode=0750,iocharset=utf8,nofail,_netdev 0 0
sudo mount -a
```

14. Security Checklist

- Use Cloudflare proxy for @ and www DNS records.
- Expose only ports 80 and 443 to the internet.

- Keep Synology LAN-only; never port-forward DSM, SMB, rsync, or NFS.
- Disable SSH password login after SSH key login is confirmed.
- Use Fail2ban.
- Use strong unique passwords for PostgreSQL, Synology users, Ubuntu users, and Cloudflare/Hostinger.
- Enable 2FA on Hostinger, Cloudflare, Synology, and admin email accounts.
- Back up PostgreSQL nightly to Synology and keep at least one offsite backup copy.
- Test restores regularly, not only backups.
- Review application logs and Nginx logs weekly during launch.

15. Deployment Update Procedure

```
cd /home/ats/app
git pull
source .venv/bin/activate
pip install -r requirements.txt
sudo systemctl restart ats
sudo systemctl status ats
journalctl -u ats -n 100 --no-pager
```

16. Go-Live Verification

- <https://yourdomain.com> loads successfully.
- Applicant registration form works.
- PDF CV upload works.
- Employee login works.
- Dashboard loads and can download CVs.
- Excel export works.
- Cloudflare DNS records are proxied.
- External scan shows only ports 80 and 443 open.
- PostgreSQL 5432, Flask 5000, and Synology ports are closed externally.
- Synology backup script has produced a recent .sql.gz backup.

17. Source References

- Hostinger DNS records: <https://support.hostinger.com/en/articles/1583249-how-to-manage-dns-records-at-hostinger>
- Hostinger A records: <https://support.hostinger.com/en/articles/4468886-how-to-add-and-remove-a-records-in-hpanel/>
- Docker on Ubuntu: <https://docs.docker.com/engine/install/ubuntu/>
- Unicorn deployment: <https://docs.gunicorn.org/en/19.9.0/deploy.html>
- Cloudflare Free plan: <https://www.cloudflare.com/plans/free/>
- Cloudflare Universal SSL: <https://developers.cloudflare.com/ssl/edge-certificates/universal-ssl/>
- Synology shared folders: https://kb.synology.com/en-af/PAS/help/PAS/AdminCenter/file_share_create
- Synology rsync: https://kb.synology.com/en-us/DSM/help/DSM/AdminCenter/file_rsync?version=7